OWASP

Open Web Application Security Project

Abuse Cases Definition

Story & overview of a possible approach.

May 2019

About me

Dominique Righetto alias *righettod* from Luxemburg

- Developer since 2003 (mainly around Java and .Net).
- AppSec enthusiast and OWASP junkie since 2011.
- Senior AppSec consultant @ excellium-services.com.
- Co-Leader the OWASP Cheat Sheet Series project.

- Presentation content is based on: https://www.owasp.org/index.php/Abuse_Case_Cheat_Sheet

@righettod

https://www.righettod.eu

dominique.righetto@gmail.com

dominique.righetto@owasp.org

OWASP Open Web Application Security Project

Agenda

- Local context
- Motivation to act on the context
- The challenge
- Abuse Case?
- Description of the approach
- Conclusion

Local context

Disclaimer:

‘The elements mentioned in the local context are based on the fact observed during my professional and personal life over the last 15 years as a developer and application security guy.’

I’m fully open to discussion about all the elements of this presentation so feel free to interrupt me to ask any question ☺

Local context

- Mostly bank and insurance companies.
- Mainly .Net & Java with NodeJS, Angular, React technologies.
- Security guided by the compliance and security devices vendor lobbies.

Local context

- Security is managed via:
  - Policies & procedures due to his banking history.
  - Installation of WAF/IDS/IPS but not customised with application specificities…
- Security awareness training to the development team has started around 2 years ago.

Local context

- Request For Proposal (RFP) from companies only contains these kinds of requirements about security expected for the build software:
  - The application must be secure.
  - The application must be compliant with the OWASP Top 10.
  - The application must follow the OWASP guidelines.
  - …

Local context

- In the specification (*waterfall*) or user story (*agile*) received by the development team:
  - There no details about the legit content of data expected to be received (characters set, encoding, min/max size…).
  - There is no flag to indicate if the data manipulated are considered sensitive or exposed to GDPR regulation.
  - There are no details about protection to implements.

Motivation to act on the context

- From a personal point of view:
  - Pissed to develop and use insecure software.
  - Want to stop shaming/pressure on the development team by allowing them to have all the elements needed/possible to include the security in their projects.
  - Erase any smile on the face of a security auditor during the ‘pre-production’ penetration test.

Motivation to act on the context

- From a professional point of view:
  - Promote **pragmatic security** at application level.
  - Allow security level of an application to become:
    - ***Identifiable***: Know against which attacks app is protected or not.
    - ***Verifiable***: Validate that protections are effective or not at any moment thanks to automated testing.
    - ***Measurable***: Ability to evaluate the quality of an app from a security point of view across audit on times.

The challenge

- Need to change the usual process of creation of the specification or user story of the business features.

- Need to convince business people of the importance to clearly identify attacks from which the business features must be protected.

The challenge

- Find a way that:
  - It is ‘easy’, quick and pragmatic to use.
  - Bring really usable information to the development team in the specification received.
  - Allow tracking of protections implemented.
  - Use public standards like ones from OWASP.

- ‘**Abuse Case (AC)**’ usage has pop in my mind when I was reading the OWASP Open SAMM referential!

Abuse Case?

- Abuse Case can be defined as:

'A way to use a feature that *was not expected by the implementer*, allowing an attacker *to influence the feature or outcome of use of the feature* based *on the attacker action* (or input).'

- An abuse case represents then an attack on a feature.

Abuse Case?

- Abuse Case can be of two kinds:
  - **Technical**:
    - Ex: Injection like SQLi or XSS in comment input field.
  - **Business**:
    - Ex: Ability to modify arbitrary the price of an article in an online shop prior to pass an order causing the user to pay a lower amount for the wanted article.
- It’s important to identify the both kinds.

Abuse Case?

- Why identify a list of abuse cases for a feature?
  - Evaluate the business risk for each of the identified attacks in order perform a selection according to the business risk and the project/sprint budget.
  - Derive security requirements and add them into the project specification or sprint's user stories acceptance criteria.

Abuse Case?

- Why identify a list of abuse cases for a feature?
  - Estimate the overhead to provision in the initial project/sprint charge that will be necessary to implement the countermeasures.
  - Allow the project team to define the countermeasures and in which location they are appropriate (network, infrastructure, code…) to be positioned.

Abuse Case?

- Abuse Case definition activity described in the Open SAMM is good but too descriptive.

- It was then decided to use it ‘as a specification’ to build a small ground-oriented approach to define abuse cases that fulfils the described challenges…

Description of the approach

- Created with a bunch of close friends with which I work on AppSec projects on Dec 2017.
- Applied on customer's agile and waterfall projects since Jan 2018.
- Shared with the OWASP community in Sept 2018 to share our win/fail and initiative…

Description of the approach

- The approach is based on the execution of a workshop in which different specific profiles of people are needed.

- The output of the workshop is a finalised list of abuse cases for all business features that will be implemented in project/sprint.

Description of the approach

- The workshop is performed at a time that depends on the project execution mode:
  - **Agile:** The definition workshop must be made after the meeting in which User Stories are associated to a Sprint.
  - **Waterfall:** The definition workshop must be made when business features to implements are identified and known by the business.

Description of the approach

- The workshop execution time is important:
  - The list of business features to implements in the project/sprint must be fixed in order to be sure to cover all features in the workshop.
  - Details of the business features must be known in a way allowing business people to provide explanation about them (data processed, flow…).

Description of the approach

- The profiles of people needed for the workshop are the following:
  - A business analyst.
  - A risk analyst.
  - An offensive guy.
  - A technical leader of the project.

Description of the approach

- **Business analyst**:
  - Will be the business key people that will describe each feature from a business point of view.
  - Must have a good knowledge of the business features requested to be implemented.
  - Must be able to explain how the business feature behave (data processed, flow, business regulation issues…).

Description of the approach

- **Risk analyst**:
  - Will be the company's risk personnel that will evaluate the business risk from a proposed attack.
  - Must be able to determine the real impact severity for the company of the abuse of a business feature.
  - Raise warning about norms (GDPR, ISO…) issues.

Description of the approach

- **Offensive guy**:
  - A Pentester or an Application Security guy with offensive mindset.
  - Will be the attacker that will propose all attacks that he can perform on the business feature that will be presented to him.
  - Must be able to provide information to the *Risk analyst* to help to determine the business impact.

Description of the approach

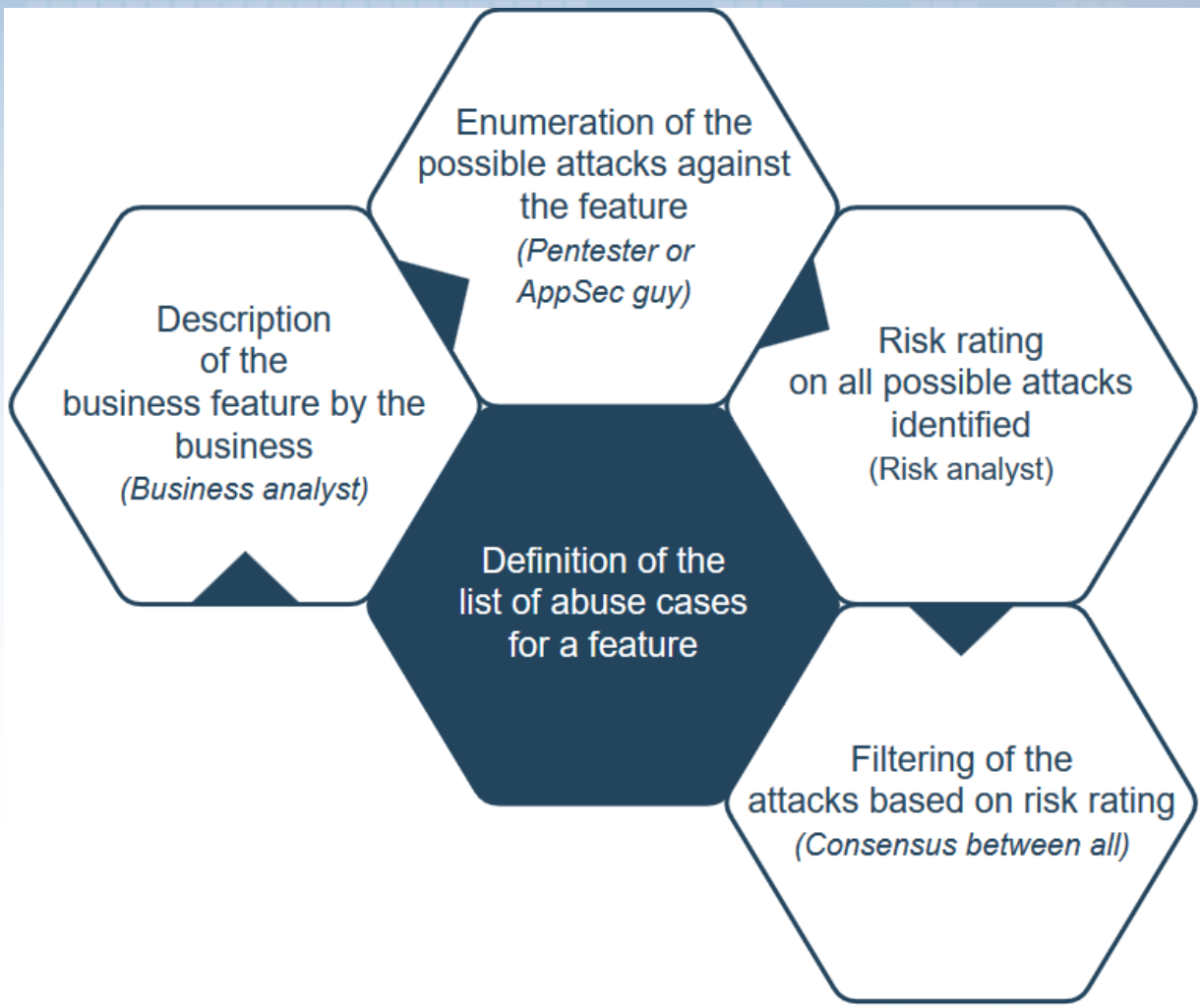
- **Offensive guy**:
  - If the company does not have this profile then it is possible to ask an intervention of an external specialist.
  - If possible, include 2 offensive guys (ex: 1 Pentester + 1 AppSec) in order to increase the number of possible attacks that will be identified and considered.

Description of the approach

- **Technical leader of the project**:
  - Will be the project technical people and will allow technical exchange about attacks and countermeasures identified during the workshop.
  - Must have a good knowledge of the technical design and stack considered for the implementation in order to identify existing/potential countermeasures.

Description of the approach

- Overview of the execution flow of the workshop →



Description of the approach

Step 1: *Preparation of the workshop*

- Create a new Microsoft Excel file (you can also use Google Sheets or any other similar software).
- Add these sheets: Features, Countermeasures, Abuse Cases.

Description of the approach

Step 1: *Preparation of the workshop*

- Features:
  - Will contain a table with the list of business features planned for the workshop.
  - This sheet is mandatory.

Description of the approach

***Step 1:** Preparation of the workshop*

- Countermeasures:
  - Will contain a table with the list of countermeasure possible (light description) imagined for the abuse cases identified.
  - This sheet is not mandatory but it can be useful to know if, for an abuse case, a fix is easy to implement and then can impact the risk rating.

Description of the approach

Step 1: *Preparation of the workshop*

- Abuse Cases:
  - Will contain a table with all abuse cases identified during the workshop.
  - This sheet is mandatory.

Description of the approach

Step 1: *Preparation of the workshop*

- Features sheet with content example:

Feature unique ID	Feature name	Feature short description
FEATURE_001	DocumentUploadFeature	Allow user to upload a document along a message.

Description of the approach

***Step 1:** Preparation of the workshop*

- Countermeasures sheet with content example:

Countermeasure unique ID	Countermeasure name	Countermeasure help/hint
DEFENSE_001	Validate the uploaded file by loading it into a parser.	Use advice from the OWASP Cheat Sheet about file upload.

Description of the approach

Step 1: *Preparation of the workshop*

- Abuse Cases sheet with content example:

Abuse case unique ID	Feature ID impacted	Abuse case's attack description	Attack referential ID (if applicable)	CVSS V3 risk rating (score)	CVSS V3 string	Kind of abuse case	Countermeasure ID applicable	Handling decision (*To Address* or *Risk Accepted*)
ABUSE_CASE_001	FEATURE_001	Upload Office file with malicious macro in charge of dropping a malware	CAPEC-17	HIGH (7.7)	CVSS:3.0/AV:N/AC:H/PR:L/UI:R/S:C/C:N/I:H/A:H	Technical	DEFENSE_001	To Address

Description of the approach

Step 1: *Preparation of the workshop*

- Usage of a unique ID for *Features*, *Countermeasures* and *Abuse Cases* are important and useful.
- Allow the creation of dictionaries of countermeasures and abuses cases that can be reused across projects/sprints.

Description of the approach

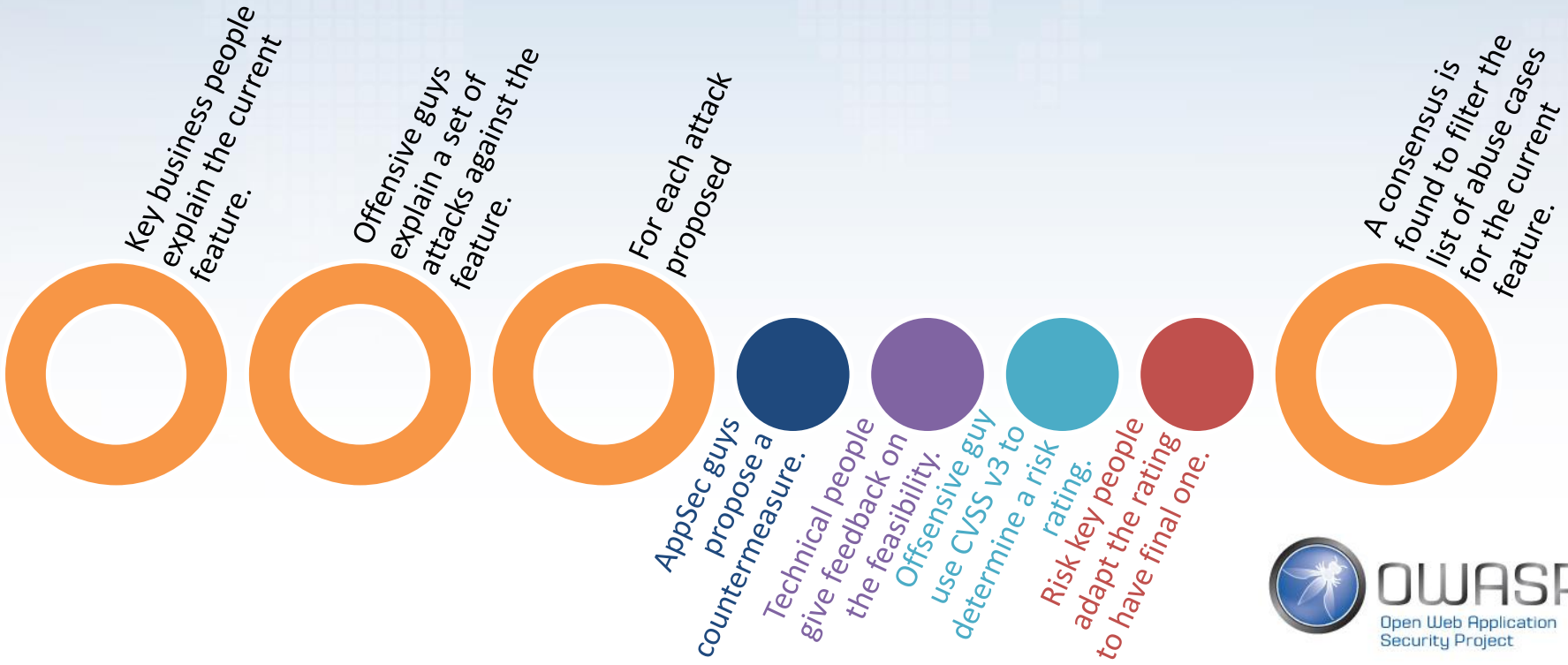
Step 1: *Preparation of the workshop*

- Allow the tracking of the handling of the abuse cases in the different components of the project/sprint (code, design and documentation).
- See further in the presentation about this topic.

Description of the approach

Step 2: *During the workshop*

- For each feature, this process is applied:



OWASP
Open Web Application Security Project

Description of the approach

Step 2: *During the workshop - Remark*

- If the presence of offensive guys is not possible then the following references can be used:
  - OWASP Automated Threats to Web Applications.
  - OWASP Testing Guide and Mobile Testing Guide.
  - Common Attack Pattern Enumeration and Classification (CAPEC).

Description of the approach

Step 3: *After the workshop*

1. Specifications or the User Stories must now include the associated abuse cases as Security Requirements or Acceptance Criteria.
2. Overhead in terms of charge/effort to take into account the countermeasures must be evaluated.

Description of the approach

***Step 4:** During impl. – AC handling tracking*

- Design, documentation and code level:
  - Add a marker like '`This design tackle abuse case [AC_ID]`'.
- About code level:
  - Annotation like `@AbuseCase(ids={'[AC_ID]'})` can be used into Developer IDE.
  - Help code reviewers to spot countermeasures impl.

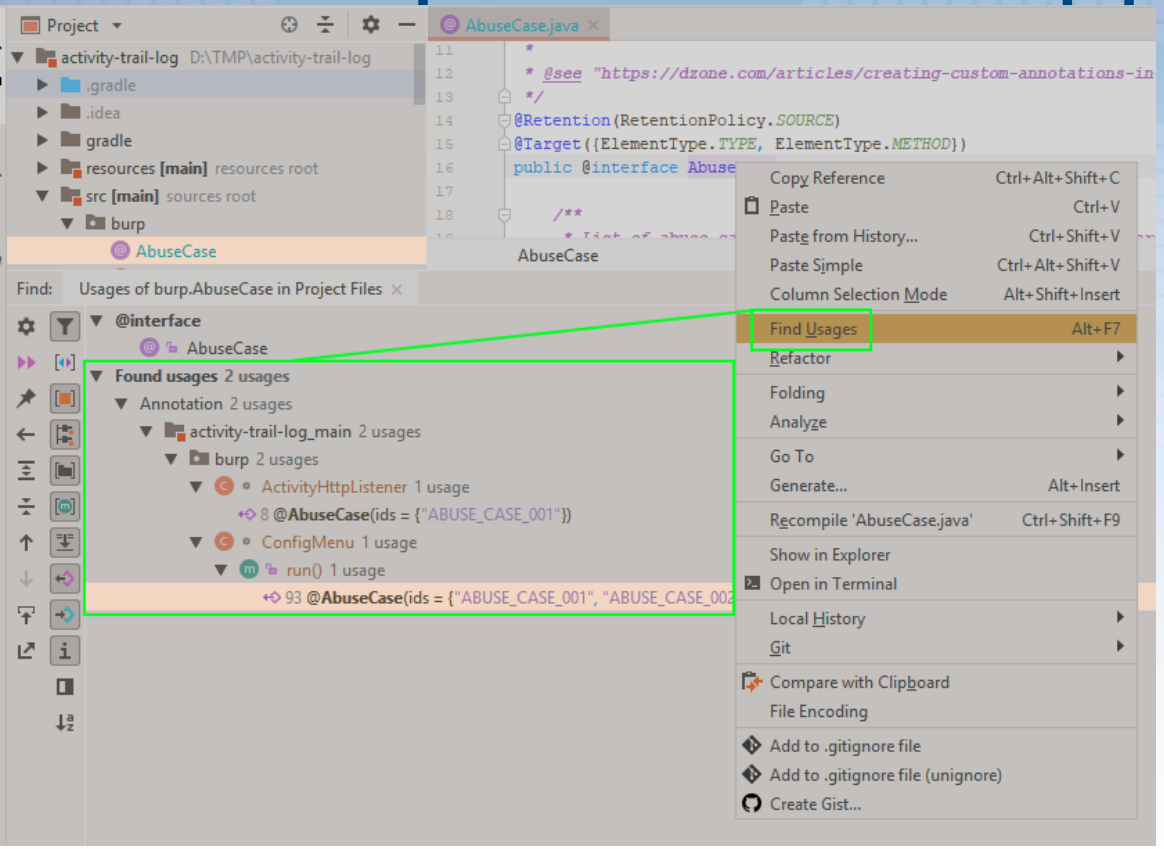
Description of the approach

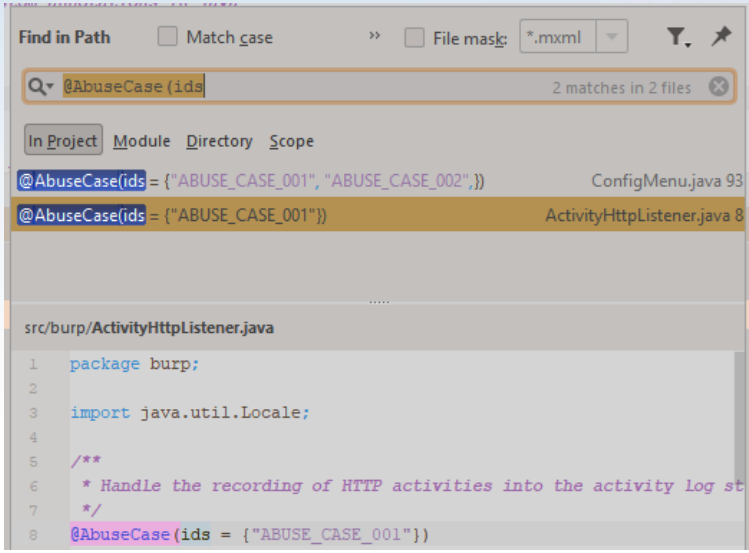
```java
/**
 * Annotation to mark and identify abuse cases in a project source code.
 * Allow to be placed on a class or a method.
 *
 * @see "https://dzone.com/articles/creating-custom-annotations-in-java"
 */
@Retention(RetentionPolicy.SOURCE)
@Target({ElementType.TYPE, ElementType.METHOD})
public @interface AbuseCase {

    /**
     * List of abuse cases identifiers that are handled by the target of the annotation
     */
    String[] ids() default "";
}
```

```java
/**
 * Handle the recording of HTTP activities into the activity log storage.
 */
@AbuseCase(ids = {"ABUSE_CASE_001"})
class ActivityHttpListener implements IHttpListener {
```

```java
    /**
     * Build the options menu used to configure the extension.
     */
    @Override
    @AbuseCase(ids = {"ABUSE_CASE_001", "ABUSE_CASE_002",})
    public void run() {
```

Description of the approach






```
$ grep -r "@AbuseCase" *
src/burp/ActivityHttpListener.java:@AbuseCase(ids = {"ABUSE_CASE_001"})
src/burp/ConfigMenu.java:    @AbuseCase(ids = {"ABUSE_CASE_001", "ABUSE_CASE_002",})
```

Description of the approach

Step 5: *During impl. – AC handling validation*

- As abuse cases are defined then it is possible to put in place validations to ensure:
  - All the selected abuse cases are handled.
  - An abuse case is correctly handled.

Conclusion

- Abuse cases can be used to clearly identify against which attacks the application must defend.
- Allow to define clearly the level of security wanted.
- Allow to identify the cost/effort that must be added for the level of security wanted.
- Provide to DEV team the real information needed to implement the defensive measures.
- Allow to do pragmatic AppSec!

I want to thanks you…

- My following buddies that helping me to build, run and share this approach:
  - Julien Ehrhart: @JulienEhrhart
  - Benoit Chenal: @Lemordac

- All of you for attending this talk and allow me this amazing sharing opportunity 

OWASP
Open Web Application Security Project

Idea for the future…

- Implement a single page application (client side only) dedicated to apply the presented approach and allow consolidation of abuse cases and countermeasures.

Q&R

OWASP
Open Web Application Security Project